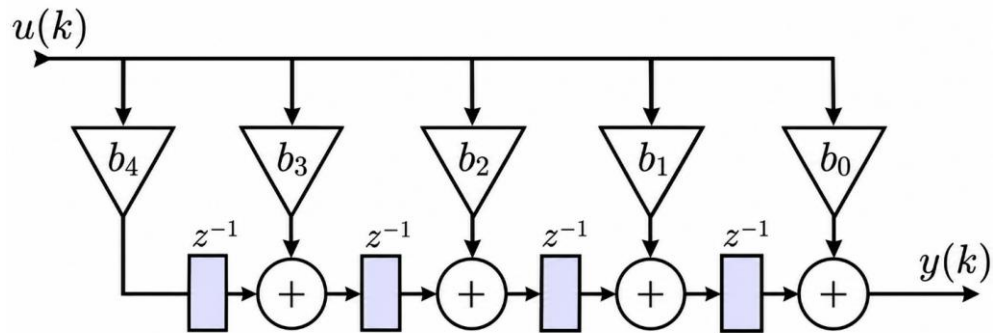


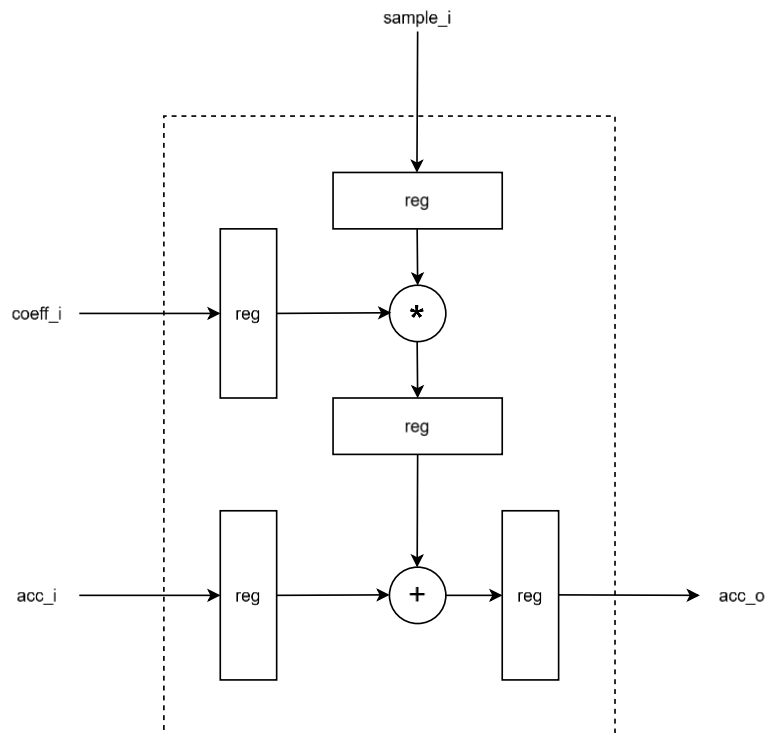
## Arhitektura sistema

Korišćena je transponovana direktna forma filtra sa dodatno ubačenom protočnom obradom (slika 1).



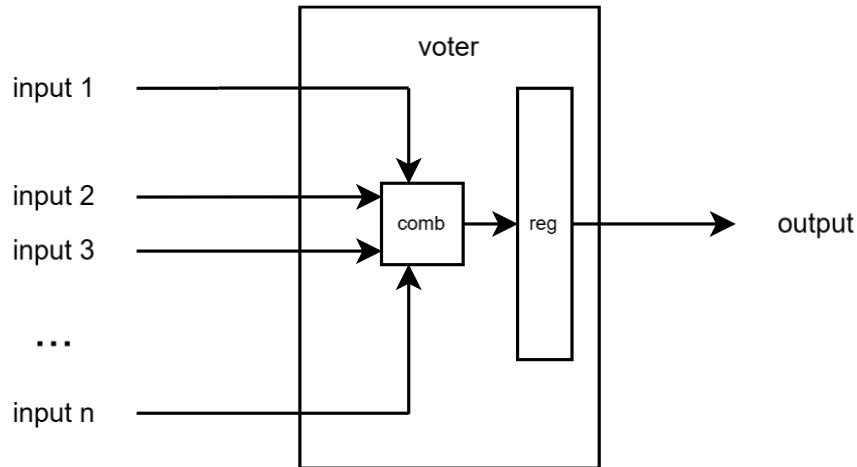
Slika 1. Transponovana direktna forma fir filtra za filter stepena 4

U osnovi sistema koriste se MAC (eng. *multiply and accumulate*) jedinice. Implementacija ove jedinice nalazi se u datoteci `mac_unit.vhd`, a njena šema je prikazana na slici 2. MAC jedinica unutar sebe ima tri faze protočne obrade i mapira se na DSP resurse FPGA čipa.



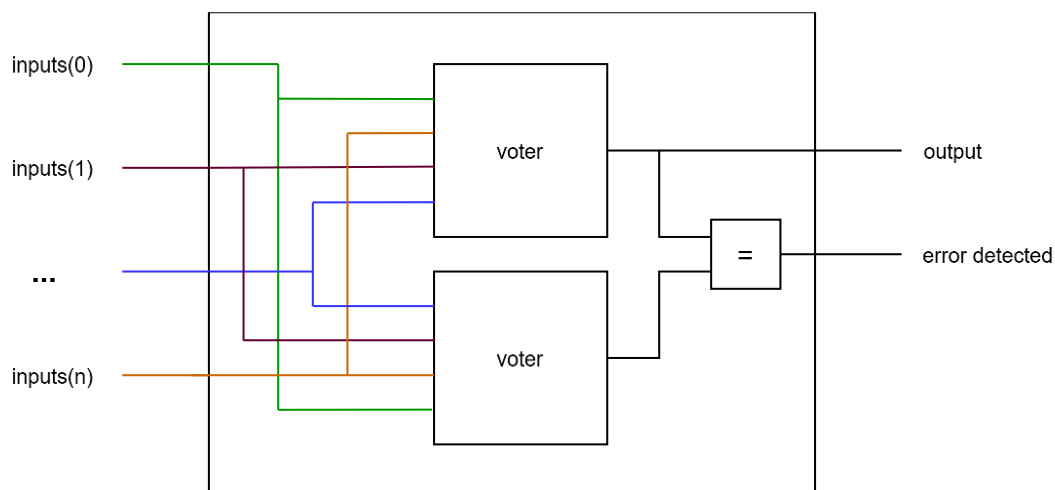
Slika 2. Implementacija MAC jedinice

Na MAC jedinicu primenjena je tehnika N modularne redundanse. Zbog toga što alat za sintezu spaja identične logičke grane implementiranog dizajna u jednu da bi se uštedelo na resursima, unutar MAC jedinice koristi se `soft` vrednost atributa `keep_hierarchy` da bi se ovaj proces (eng. *decloning*) onemogućio. `soft` vrednost dopušta optimizaciju propagacije konstanti kroz modul uz očuvanje hijerarhije i preporuka je koristiti je umesto `yes / true` gde je to moguće, iz kog razloga je ona izabrana.



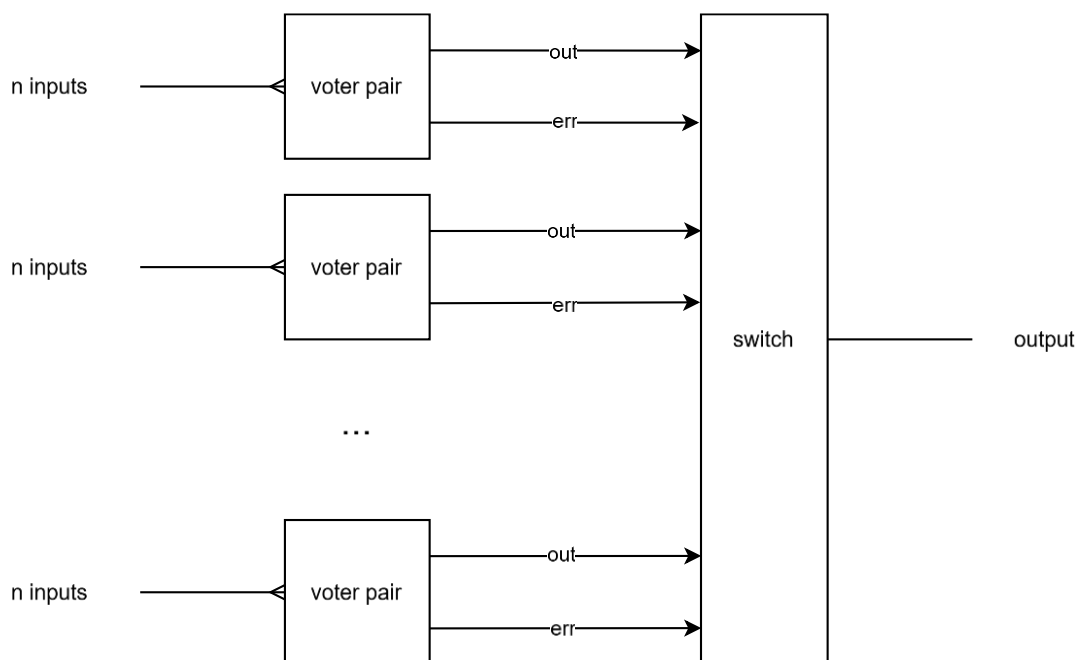
Slika 3. Šema implementacije glasača

Tip glasača koji je implementiran je većinski registrovani glasač (slika 3). Njegov kombinatorni deo računa koji ulaz ima većinu i propušta njegovu vrednost na ulaz izlaznog registra. Ukoliko ne postoji većina propušta se prvi ulaz. Glasač je registrovan iz razloga što se i na njega primenjuje `soft` vrednost `keep_hierarchy` atributa, a AMD preporučuje da se moduli na koje se ovaj argument primenjuje budu registrovani (na ovaj način se dobijaju maksimalne performanse sistema). Implementacija se nalazi u datoteci `majority_voter.vhd`.



Slika. 4 Šema implementacije para glasača

Na glasač primenjena je tehnika *pair and a spare*. Dva glasača rade u paraleli, a ako se jedan od njih pokvari javlja se greška (slika 4). Svaki glasač dobija izlaze svake MAC jedinice (slika 5). Zbog toga što će par glasača biti spojen u jedan prilikom sinteze, unutar instance glasača deklariše se `keep_hierarchy` atribut vrednosti `soft`.

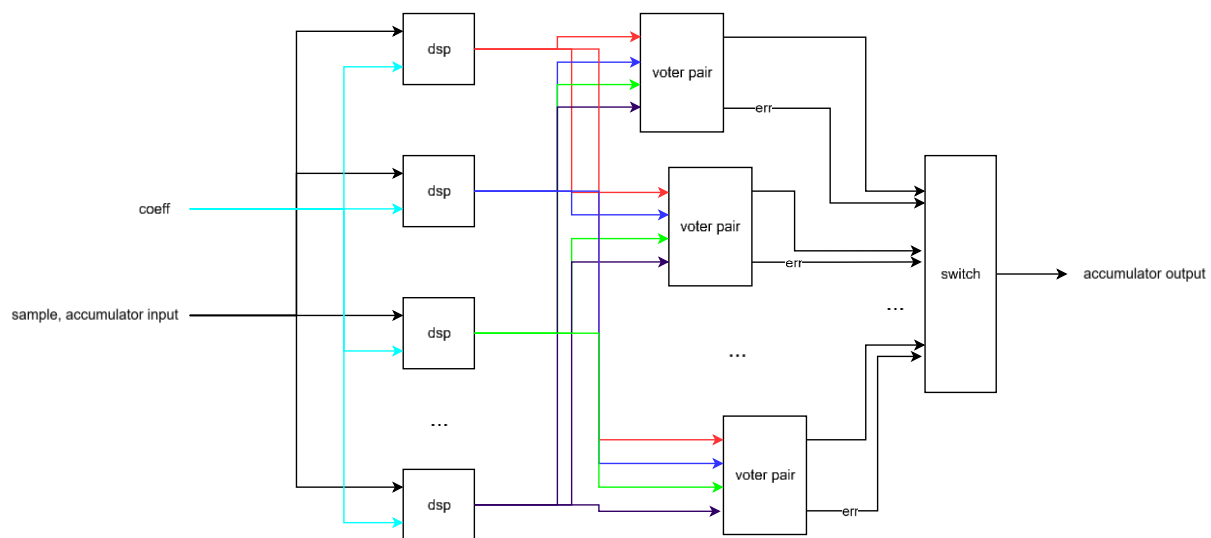


Slika 5. Pair and a spare glasač, implementacija

Prekidač na izlazu bira ulaz bez greške i prosleđuje ga na izlaz. Parova glasača ima proizvoljno mnogo, u zavisnosti od stepena tolerancije sistema na greške. Implementacija para glasača

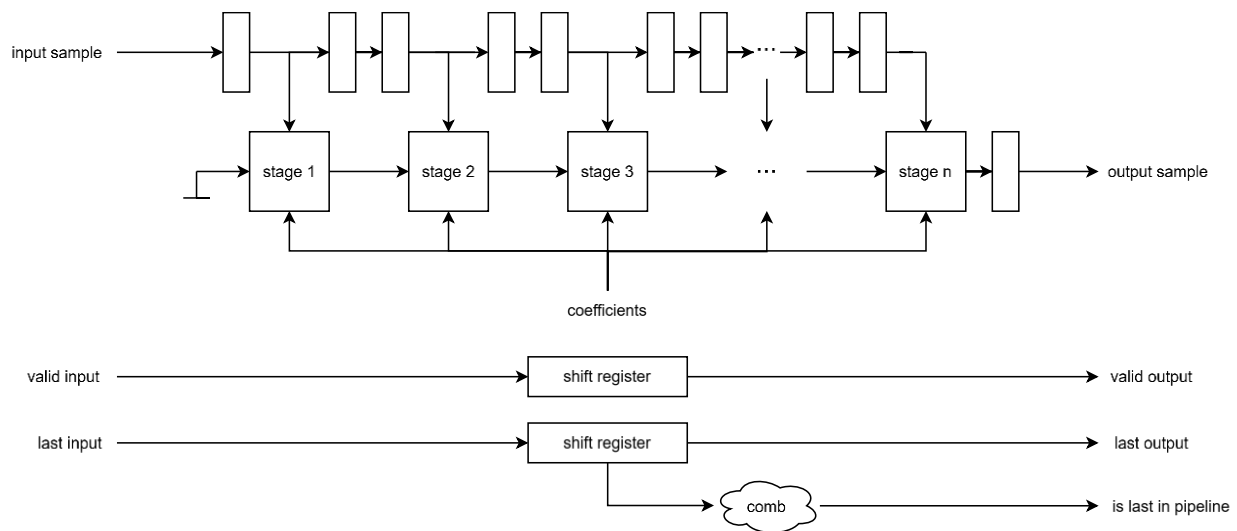
nalazi se u datoteci `voter_pair.vhd`, a implementacija celog pair and a spare glasača u datoteci `pair_and_a_spare_voter.vhd`.

Na slici 6 prikazan je jedan stepen filtra. Step se sastoji iz više MAC jedinica (dsp na slici) povezanih na pair and a spare glasač sa slike 5. Na slici je prikazana i unutrašnjost pair and a spare glasača (parovi glasača i prekidač). Hardverska implementacija nalazi se u datoteci `fir_stage_ft.vhd`.



Slika 6. Jedan stepen filtra, šema

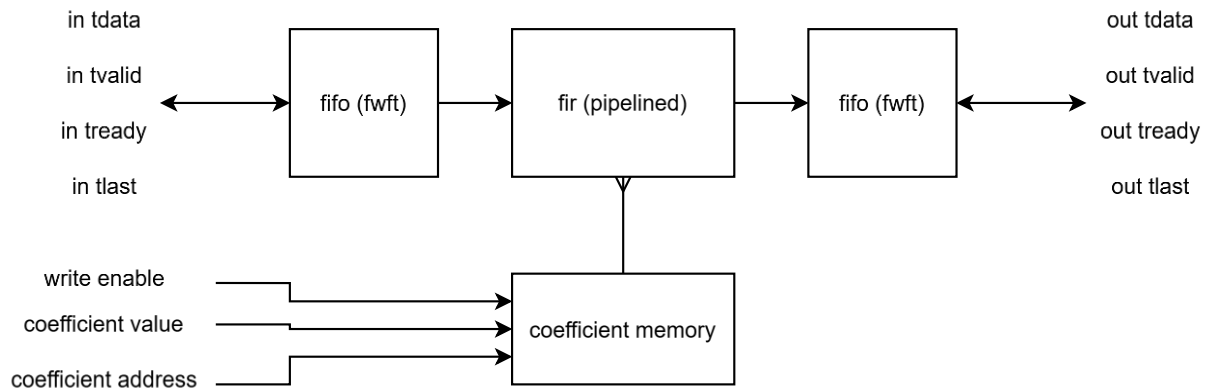
Broj stepena FIR filtra za jedan je veći od njegovog reda. Koristeći hardversku implementaciju stepena filtra sa slike 6, implementiran je ceo filter, dok je na višem nivou hijerarhije dodata podrška za AXI interfejs. Šema implementiranog filtra može se videti na slici 7, a nalazi se u datoteci `fir_filter_ft.vhd`.



Slika 7. Šema implementacije FIR filtra (bez AXI interfejsa)

Dodat je registar na ulaz i izlaz filtra da bi dizajn radio na višim frekvencijama, jer na višem nivou hijerarhije ima kombinacione logike. Neophodno je da se ulazni odbirak zakasni dva puta pre ulaska u svaki naredni stepen filtra (osim prvog), zato što su izlaz akumulatora i izlaz glasača registrovani. U suprotnom ne bi se dobila željena prenosna funkcija filtra. Koeficijenti filtra dovode se na odgovarajuće stepene sa top modula. Takođe je neophodno zakasniti tvalid i tlast signale sa ulaza da bismo znali koji je odbirak validan / poslednji. U ove svrhe koriste se dva šift registra, iste onolike dubine kolika je dubina pipeline-a filtra. Takođe postoji izlaz `last_in_pipeline`, neophodan top modulu pri pražnjenju pipeline-a na kraju AXI prenosa.

Modul na najvišem nivou hijerarhije (tzv. top modul) implementiran je u datoteci `axi_wrapper.vhd`. Šema modula prikazana je na slici 8. Modul je zadužen za rukovanje AXI protokolom i upis koeficijenata u memoriju. AXI magistrala dodata je na filter postavljajući FIFO bafere na njegove ulaze i izlaze. Korišćen je FWFT (eng. *First Word Fall Through*) tip FIFO bafera, zbog svoje brzine. Detalji implementacije nalaze se u izvornoj datoteci `fifo.vhd`.



Slika 8. Šema top (AXI omotač) modula

U ulazni i izlazni bafer upisuju se samo validni podaci. Oba bafera imaju izlazne indikacione signale full i empty. Ne sme se dozvoliti da u FIR blok uđu nevažeći podaci jer njegov izlaz zavisi od redosleda i vremena propuštanja ulaznih odbiraka. Zbog toga se ceo pipeline filtra zaustavlja ukoliko je ulazni bafer prazan. Pipeline takođe treba zaustaviti ako je izlazni bafer pun. Da podaci ne bi ostali zaglavljani unutar pipeline-a na kraju prenosa, oni se izbacuju u izlazni bafer (ukoliko u njemu ima mesta) tako što se na ulaz filtra šalju sve nule. MAC jedinice u ovom slučaju rade samo pomeranje. U bafere se osim ulaznih ili izlaznih podataka upisuje i tlast vrednost, radi generisanja izlaznog tlast signala. Ulazni tready i izlazni tvalid signal lako se izračunavaju na osnovu stanja bafera. U ulazni bafer upisuje se samo ukoliko je došlo do handshake-a na ulazu, dok se iz izlaznog bafera čita jedino ukoliko je došlo do handshake-a na izlazu.

## Simulacija i verifikacija

Napravljen je jednostavan test benč u svrhe provere ispravnosti implementiranog hardverskog sistema (datoteka `fir_filter_ft_tb.vhd`). Test prima tekstualne datoteke u kojima je definisana ulazna pobuda, koeficijenti filtra i očekivani izlazi filtra. Ove datoteke generisane su korišćenjem MATLAB programskog paketa (datoteka `fir.m`). Ulazna pobuda je zbir dve sinusoidne – brze i spore. Filtar za test je niskopropusni. Potrebno je imati Signal Processing Toolbox, Fixed-Point Designer i DSP System Toolbox pakete instalirane u okviru MATLAB-a da bi se program uspešno pokrenuo. Testiran je filtar reda 5, ulazne širine 24 i izlazne širine 32. U top modulu

(`axi_wrapper.vhd`) potrebno je podesiti broj grešaka na koji je dizajn otporan pre pokretanja simulacije, kao i širinu ulaznih i izlaznih podataka i red filtra. Parametri moraju da se poklapaju sa parametrima u `firm` datoteci da bi simulacija davala verodostojne rezultate. Program za generisanje datoteka za simulaciju simulira filter sa istom širinom ulaznih odbiraka i akumulatora kao i VHDL model, te su generisane očekivane vrednosti u decimalu tačne sa izlazima iz (ispravnog) HDL modela.

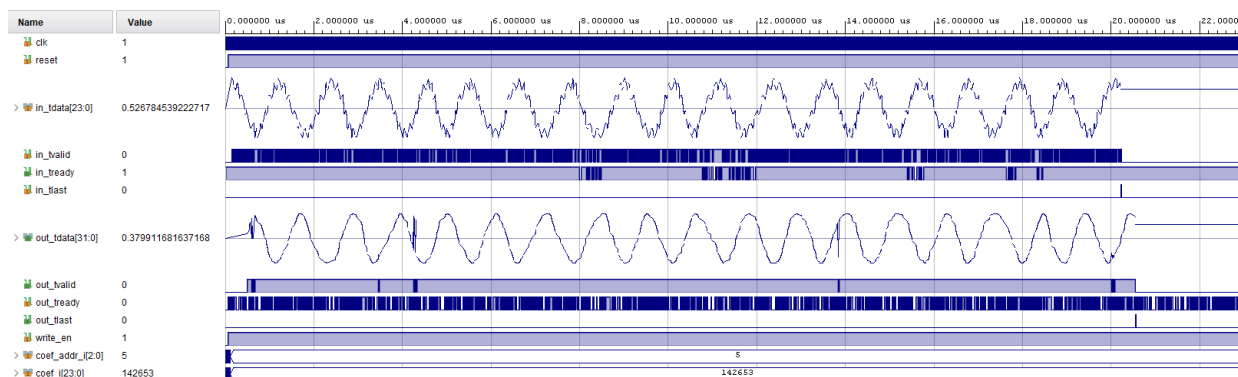
S obzirom na to da su i MAC jedinice i glasači otporni na greške, radi jednostavnosti implementirano je da se njihov broj menja u zavisnosti od podešenog broja grešaka na koji je sistem otporan (parametar `MAX_FAULT_TOLERANCE`). Pri pokretanju simulacije (korišćena isključivo simulacija tipa behavioral) pokreće se TCL skripta `inject_fault.tcl`. Skripta ubacuje onoliko kvarova koliko sistem maksimalno može da podnese. Kvarovi se ubacuju na izlaze MAC jedinica i izlaze glasača svakog od stepena filtra. Koriste se posebne hex vrednosti za ubačene greške / vrednosti – za MAC jedinice to je „bad“, a za glasače „dead“. Na ovaj način u simulaciji korisnik lako vidi gde se greška nalazi. Promenom vrednosti promenljive imena `pkvvari` na `true` unutar skripte u sistem je moguće uneti jednu grešku više od tolerisanog broja grešaka. Na ovaj način moguće je videti da sistem prestaje da funkcioniše kada se unese jedna greška više od dozvoljenog broja. Ideja skripte je da unese maksimalan dozvoljeni broj grešaka u sistem, a da korisnik potom u simulaciji proveri da sistem funkcioniše pravilno bez obzira na unete greške. Jednom ubačene greške traju celu simulaciju.

Test benč napisan je tako da čita datoteku `expected.txt` (generisanu od strane MATLAB programa) prilikom handshake-a na izlazu sistema, i upoređuje izlazne vrednosti u tom trenutku sa očekivanim vrednostima. Ukoliko dođe do neslaganja ispisuje se odgovarajuća poruka u TCL konzoli. Generiše se pobuda koja poštuje AXI protokol sa nasumičnim obaranjem tvalid signala na logičku nulu – simulira se pauziranje prenosa. Izlazni `tready` takođe ima nasumičnu vrednost – simulira se situacija kada prijemnik nije spreman da primi podatke od našeg modula. Na ovaj način modul koji se testira je maksimalno pobuđen.

Takođe se u posebnom procesu proverava da li izlazni podaci poštuju AXI protokol, tj. prijavljuje se greška u TCL konzoli ukoliko se neki od izlaznih signala promeni pre nego što se

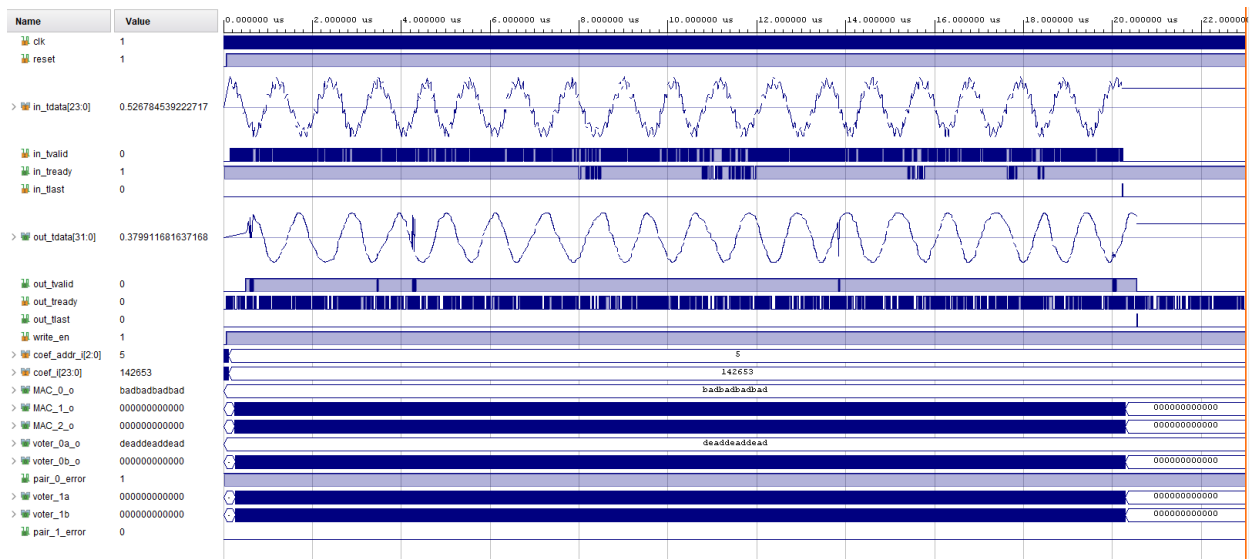
handshake desio, što nije dozvoljeno jednom kada je validan signal postavljen na izlaz. Osim ovih provera test benč proverava i da li se tlast na izlazu aktivirao u odgovarajućem trenutku.

Generisana je pobuda od 1000 odbiraka da bismo bili sigurni da sistem radi kako treba, tj. da bi se simulirao granični slučaj kada se izlazni i ulazni baferi napune do kraja. Rezultat simulacije prikazan je na slici 9.



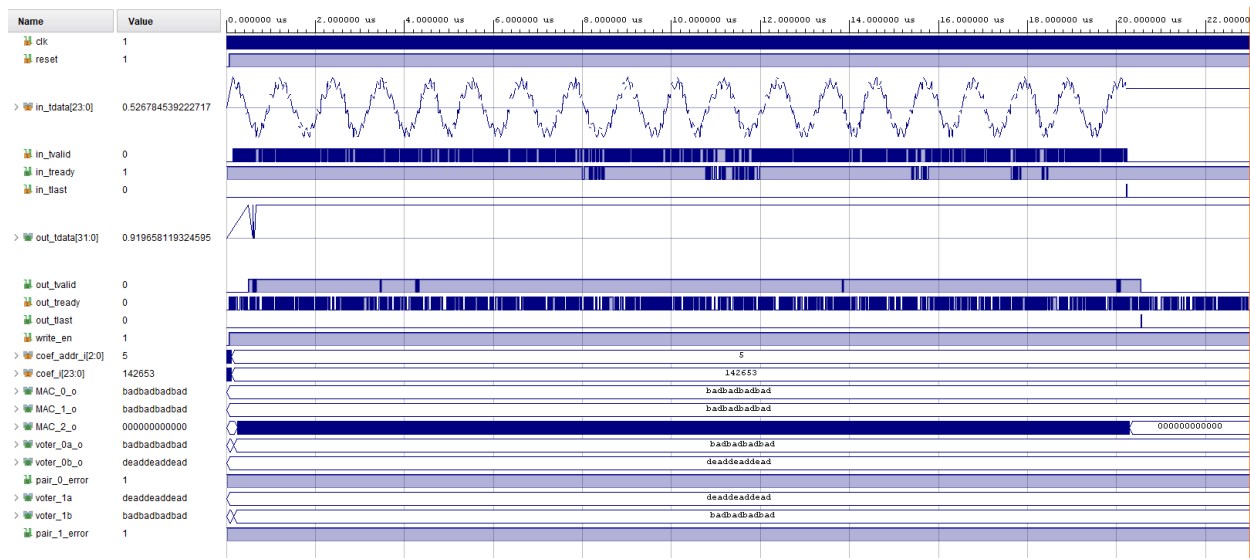
Slika 9. Talasni oblici nakon pokretanja simulacije

Korišćenjem scope i objects prozora moguće je videti izlaze iz svakog od redundantnih modula. Primetiti „bad“ i „dead“ sekvence koje su forsirane TCL skriptom. Na slici 10 prikazan je izlaz iz MAC modula i glasača jednog od stepena filtra za slučaj da je sistem otporan na jednu grešku (signali preimenovani radi preglednosti). Uočiti da sistem i dalje radi kako je predviđeno iako je ubačena jedna greška u MAC jedinicu i jedna u glasač. TCL skripta automatizuje ovaj proces ubacivanja grešaka za proizvoljnu toleranciju modela na greške (može biti i 0).



Slika 10. Rezultati simulacije, prikaz forsiranih grešaka u prozoru (pokvari je false)

U TCL konzoli nema ni jednog ispisa greške, te se može zaključiti da sistem funkcioniše po specifikaciji. Ispitivanjem talasnih oblika u simulaciji može se doći do istog zaključka – podaci na izlazu su oni očekivani, stabilni su dok se ne desi handshake na izlazu i nisu ispravni ukoliko izlazni tvalid nije logičko 1.



Slika 11. Rezultati simulacije za vrednost true promenjive pokvari iz tcl skripte

Pokretanjem simulacije sa forsiranjem jedne greške više od dozvoljenog broja dobijaju se rezultati sa slike 11. TCL konzola preplavljena je porukama grešaka i vizuelno se primeti da

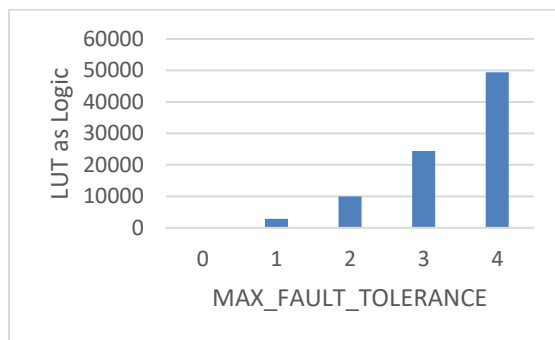
sistem ne daje očekivani izlaz. Međutim, sistem je uspešno detektovao da postoje dve greške (pair\_0\_error i pair\_1\_error vrednosti sa slike 11) iako nije uspeo da ih otkloni, što je osobina pair and a spare tehnike otpornosti na greške.

## Analiza resursa i performansi

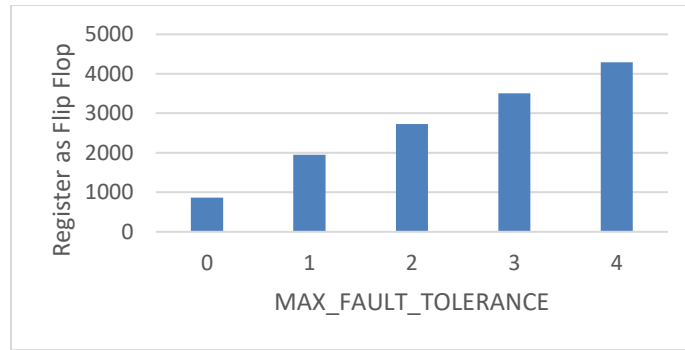
Hardverski sistem implementiran je na FPGA uređaju xc7k160tffg676-2 (familija Kintex-7), u svrhe evaluacije utrošenih resursa i vremenske analize sistema. Pokrenuta je implementacija za sistem bez redundanse (otporan na 0 grešaka) i sistem otporan na 1, 2, 3 i 4 greške. Izveštaji nakon uspešno izvršene implementacije nalaze se unutar fascikle reports/utilization a sumirani su u tabeli 1. Menja se utrošenost LUT i DSP elemenata, dok utrošenost IOB i BUFGCTRL resursa ostaje nepromenjena (što je očekivano s obzirom na to da se broj ulaza i izlaza ne menja sa povećanjem redundantnosti sistema). Na graficima 1, 2 i 3 prikazana je promena utrošenosti LUT elemenata za logiku, LUT elemenata za flip-flobove i DSP elemenata u zavisnosti od parametra MAX\_FAULT\_TOLERANCE, respektivno.

Tabela 1. Utrošenost resursa za različite vrednosti parametra MAX\_FAULT\_TOLERANCE

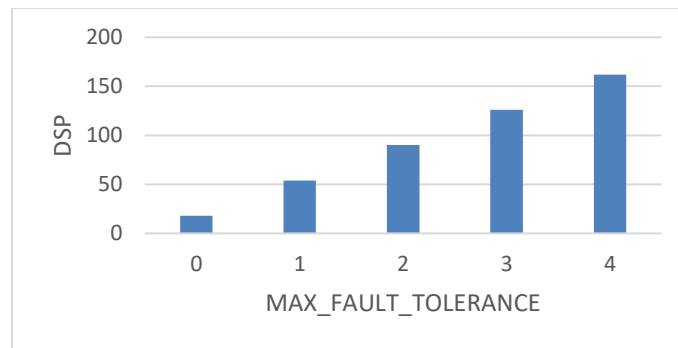
| MAX_FAULT_TOLERANCE | LUT as Logic | LUT as Distributed RAM | Register as Flip Flop | DSP | Bonded IOB | BUFGCTRL |
|---------------------|--------------|------------------------|-----------------------|-----|------------|----------|
| 0                   | 46           | 42                     | 866                   | 18  | 92         | 1        |
| 1                   | 2837         | 42                     | 1950                  | 54  | 92         | 1        |
| 2                   | 9998         | 42                     | 2730                  | 90  | 92         | 1        |
| 3                   | 24397        | 42                     | 3510                  | 126 | 92         | 1        |
| 4                   | 49402        | 42                     | 4290                  | 162 | 92         | 1        |



Grafik 1. Utrošenost LUT-ova na logiku u zavisnosti od parametra MAX\_FAULT\_TOLERANCE



Grafik 2. Utrošenost LUT-ova na flip-flopove u zavisnosti od parametra MAX\_FAULT\_TOLERANCE

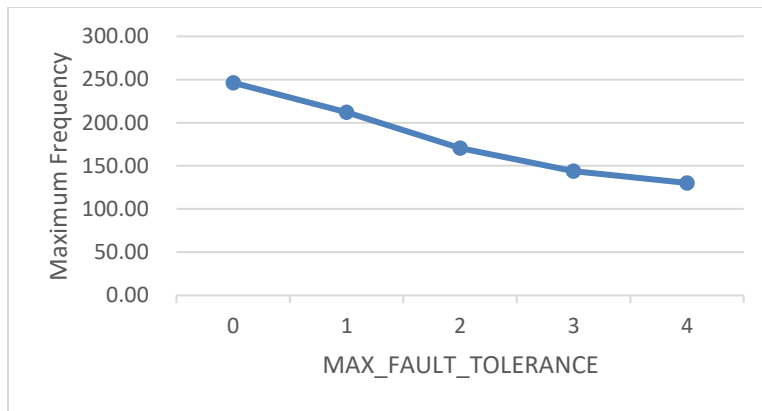


Grafik 3. Utrošenost DSP elemenata u zavisnosti od parametra MAX\_FAULT\_TOLERANCE

U tabeli 2 izlistani su rezultati vremenske analize nakon što je izvršena implementacija na FPGA čip u alatu Vivado. Vrednost WNS [ns] može se očitati i iz priloženih izveštaja u fascikli reports/timing. U constraints datoteci `timing.xdc` definisana perioda takta je 8 ns odakle je moguće izračunati maksimalnu frekvenciju sistema. Kao što je bilo i očekivano, frekvencija značajno pada kako se u sistem uvode novi redundantni moduli, što je prikazano i grafički na grafiku 4.

Tabela 2. Rezultati vremenske analize nakon implementacije

| MAX_FAULT_TOLERANCE | WNS   | Maximum Frequency |
|---------------------|-------|-------------------|
| 0                   | 3.94  | 246.31            |
| 1                   | 3.286 | 212.13            |
| 2                   | 2.135 | 170.50            |
| 3                   | 1.044 | 143.76            |
| 4                   | 0.32  | 130.21            |



Grafik 4. Promena maksimalne frekvencije sistema u zavisnosti od parametra MAX\_FAULT\_TOLERANCE